Lesson 2.3: Iterating diagnostically

Claude Certified Associate – Foundations (CCAO-F) · Module 2: Prompting & Task Execution · Lesson 3 · 17:23

Description

Even a good prompt often gets an answer that's close, but not right. Most people then use it anyway, type "make it better", or rewrite the whole prompt and never learn what fixed it. This lesson teaches a better way: iterating diagnostically. Write down what good looks like first, read the answer against that checklist, name the symptom and find its cause in the prompt, change one thing, then re-run and compare. You see where the guidance comes from: the official exam guide, Anthropic's official prep course, the Claude Platform Docs, the Claude blog and two Claude Academy courses. Then one real job runs in Claude Desktop: drafting Lesson 2.2's quick check from that lesson's published notes. The first draft passes one of five checks. One added line takes it to four, a second line to five, and the same prompt passes again in a brand-new chat. The lesson ends with one exam-style scenario, reasoned out loud, and the cue to this lesson's quick check.

Learning outcomes

By the end of this lesson you can:

1. Read an answer against your success criteria, and name the symptom and its cause.
2. Change one thing per run, re-run, and tell whether that change fixed it.
3. Choose between editing the prompt, following up, and starting fresh.

Chapters

Time	Chapter
00:00	Preview and outcomes
00:37	Why it matters
01:42	The idea: read, name, change one, re-run
03:45	Straight from the source
05:25	Hands-on: three runs, one change each
15:25	Exam angle
16:43	Recap and quick check

The idea

Criteria first. Before you read anything, write down what good looks like. Anthropic's docs call these your success criteria. Without them you're judging by feel, and "it feels off" isn't something you can fix.

The loop (this course's tool, built from the sources below):

1. **Read** the answer against your checklist.
2. **Name the symptom:** exactly what's wrong, with the line that shows it.
3. **Find the cause:** what in your prompt let that happen?
4. **Change one thing.**
5. **Re-run and compare** against the same checklist. Go around again until every check passes.

Why only one change? Change four things at once, and if something new goes wrong, you can't tell which change did it. Change one, and the difference points straight at it. One honest note: the same prompt rarely gets the same answer twice, so a prompt you'll reuse deserves more than one test.

Fix the cause, not the symptom. Causes usually hide in the parts of the prompt from Lesson 2.1 (this course's symptom map, built from the blog's troubleshooting list):

Symptom	Likely fix in the prompt
Too generic	Be specific, or add an example
Misses the point	State your goal, and why
Wrong shape	Show the shape you want
Too much at once	Split it into a chain (Lesson 2.2)
The prompt is fine	The cause may be the model or the feature (Module 7)

Three places to change things (the desk from Lesson 1.4):

- **Follow up:** reply in the same chat. Claude sees everything above, including the draft, and the desk grows.
- **Edit:** change the original prompt and send it again. In the app this starts a new **branch**: the first answer is kept, one click away, but it's no longer part of the conversation.

- **Start fresh:** a new chat with just the improved prompt.

Which one? (this course's decision rule)

- A problem in a prompt you'll reuse: **edit** it, and fix it at the source.
- A one-off tweak to a mostly good answer: **follow up**, and say what to keep.
- A chat that's off track or long, or a clean test: **start fresh**.

Straight from the source (what the lesson shows)

- **Exam guide, page 4:** "Exam items are written against these objectives." Domain 1, Prompting and Task Execution (14% of scored items), third bullet: "Iterate prompts to improve output quality". This course labels it task 1.3; the guide lists its task statements as unnumbered bullets, so 1.3 is our label, not the guide's. Refining the output itself belongs to Domain 2 (Module 3), and diagnosing prompts or outputs that still underperform belongs to Domain 7 (Module 7).
- **Anthropic's official prep course, Prompting & Task Execution:** the learning objective "Iterate prompts diagnostically to improve output quality". Diagnostically is the word this lesson is built on.
- **Claude Platform Docs, Prompt engineering overview:** the guide assumes three things up front: a clear definition of the success criteria, ways to test against them, and a first draft prompt to improve. It also warns that not every failure is best solved by prompt engineering; latency and cost, for example, can be easier to improve by selecting a different model.
- **Claude blog, Best practices for prompt engineering:** a troubleshooting list (too generic, off topic or missing the point, inconsistent format, too complex, making things up), each with its fix. And the rule behind one change per run: "Start simple and add complexity only when needed. Test each addition to see if it actually improves results."
- **Claude Academy, Claude 101, The iteration mindset:** treat first drafts as starting points; give specific feedback (not just "make it shorter", but exactly what to cut); and know when to start fresh: sometimes a new chat with a clearer prompt is faster than trying to redirect.
- **Claude Academy, AI Fluency, The Description-Discernment loop** (Lesson 0.2): describe what you need, discern the quality of what you get, refine, and go again.

Key terms

- **Iteration:** improving a result in rounds: read it, change the prompt, run it again.
- **Success criteria:** what "good" means for this result, written down before you read it. This is the Docs' term.
- **Symptom and cause:** what's wrong in the answer, and what in the prompt let it happen.
- **One change per run:** change one thing at a time, so you can tell what made the difference. This is the course's rule, from the blog's "test each addition".
- **Follow-up:** a new message in the same chat. Claude sees everything above it, including the draft.
- **Branch:** editing a sent prompt starts a new branch. The old answer is kept, but it's no longer in the conversation.

What happened in the hands-on

Claude Desktop on Linux, Chat view, Sonnet 5.5 at Medium, recorded on 2026-10-05 on a real account. Every run went into an **incognito chat** (Lesson 1.4), which isn't saved and uses no memory. The material is real: Lesson 2.2's published lesson notes, about 2,500 words, attached as a file. The job is real too: the course needs a quick check after every lesson. The exact prompts, the checklist and Claude's replies are in the student kit. Every wait while Claude worked plays in real time; nothing is cut or sped up.

Step 0: the checklist, before any prompt. Five checks, written by the course author:

1. Tests a lesson outcome: a decision, not a number or a moment from the video.
2. A short work situation that asks for the most appropriate action.
3. A new situation, not the lesson's own exam scenario.
4. Fair options: one best answer, three tempting wrong ones, and the right one is not the longest.
5. True to the notes: every fact matches the lesson notes.

Run 1: the first prompt. Four labelled lines, built the Lesson 2.1 way: Task (the quick check for Lesson 2.2, three multiple-choice questions), Context (the attached notes; learners take it right after the video), Audience (beginners with no AI background) and Format (question, options A to D, the correct letter, one sentence on why). A good prompt, but it never says what the questions should test.

1. **The draft:** Q1 "What is prompt chaining?" (a definition). Q2 asks what a moment is called (the name of a term). Q3 asks for a choice, but two of its options are Lesson 2.2's own glossary job and survey-comments scenario. In Q1 and Q2 the right answer is the longest option.
2. **Score: 1 of 5.** Checks 1, 2, 3 and 4 fail; check 5 (true to the notes) passes.
3. **The cause:** nothing in the prompt says what to test, so Claude went for what's easiest, facts from the page. It did what was asked; the prompt didn't ask for the right thing. Four checks failed, but one change at a time: truth passed, so next is what gets tested (checks 1 and 2).

Run 2: edit the prompt, one new line. The problem lives in a prompt that will be reused after every lesson, so the sent message is edited (hover, pencil). It opens in place with the file still attached; the info icon says making changes creates a new branch in the conversation. One line goes in after Context:

> Test: Each question checks one of the lesson's three learning outcomes as a short work situation that asks for the most appropriate action, because the exam tests judgment, not memory of the video.

A label, what to do, and why. The Docs note that giving the reason helps, because Claude can generalize from the explanation.

4. **The branch:** arrows "2 / 2" appear under the message. The left arrow shows version 1 and its first answer; the right arrow returns. The file came through the edit.
5. **The draft:** real work situations (step 2 of a product-review chain; 40 customer emails and an FAQ page; a teammate who wants to chain a three-sentence email). But the right answer is the longest option in every question.
6. **Score: 4 of 5.** Checks 1, 2, 3 and 5 pass; check 4 fails. One line fixed three checks, because they had one cause.

Run 3: edit again, one more line. The remaining symptom: the longest option gives the answer away. The cause: the prompt says nothing about the options. The letters already vary, so the line asks only for what's needed. After Audience:

> Constraints: Keep the four options about the same length, so length never gives the answer away.

7. **Score: 5 of 5**, version "3 / 3". The status line even showed the new line at work: "balancing option lengths to avoid obvious answer clues". All options are now 59 to 68 characters. In Q1 the right answer is still the longest by three characters, close enough that length no longer points to the answer, which is what check 4 is for.
8. **No follow-up this time.** Run 3 left nothing to polish, and no flaw was invented to demonstrate one. If you only wanted one wrong option to be more tempting, that's about this set, not the prompt: reply in the same chat and say what to keep ("keep questions 1 and 3 as they are; in question 2, change only the weakest wrong option"). If you ask for the same tweak every time, it isn't a one-off; it belongs in the prompt.

The fresh test. If the prompt itself is fixed, it should pass without the chat's history. The final prompt (the four original lines plus the two added ones) was copied from the sent message, the incognito chat was closed, and a new incognito chat got the same file and the pasted prompt. No version arrows: this is the only version.

9. **The new set:** different questions (a marketing assistant's two-step chain, an office manager's meeting notes, a shop owner's three-line email), because no two runs write exactly the same set.
10. **Score: 5 of 5 from a clean start**, with fair options (in Q3 the right answer is the shortest). The fix lives in the prompt, not in the chat.

Side by side: the prompt grew by two lines, not twenty, each tested before the next. 1 of 5, then 4, then 5, and 5 again in a fresh chat.

One honest step remains. 5 of 5 means the drafts meet the checklist, not that they're finished. The author still checks every question and its reasoning against the notes, and the published Lesson 2.2 quick check is written fresh; these on-camera drafts are not reused as it.

The exam link

Domain 1, Prompting and Task Execution (14% of scored items). This lesson serves the bullet this course labels **1.3**: "Iterate prompts to improve output quality". The guide lists its task statements as unnumbered bullets; the number 1.3 is this course's label, not the guide's.

The lesson's practice scenario (original, written for this course, not from the exam). A marketing coordinator reuses one saved prompt every week to turn release notes into a customer FAQ. The answers ran long and used internal code names, so she rewrote the prompt in one go: a new role, three example answers, a word limit and a new format. Now the answers are short and clean, but one describes a feature that isn't in the release notes. What's the most appropriate next step? A: reply, "remove the made-up feature and keep everything else". B: go back to the old prompt and add her changes one at a time, re-running each against her checklist. C: switch to the most capable model. D: add "never mention features that aren't in the release notes" in capitals. The answer is **B**. Ask what changed: four things at once, so the cause is hidden. A fixes this week's copy, not the prompt she reuses. C changes the model, but the problem came with the prompt change, and she still won't know which change did it. D guesses at the symptom; capitals add pressure, not information, and if an example answer mentions an extra feature, D fights her own example. B finds the cause and leaves her a prompt she can trust.

Quick check

Before you move on, take this lesson's quick check: a few short questions, a couple of minutes. The exam-style question in this lesson, the quick checks, the module quizzes and the practice exams are all original, written for this course against the official blueprint. They are never copies of the real exam.

Sources (all checked 2026-10-05)

- **Anthropic, *Claude Certified Associate – Foundations Exam Guide*.** Version 1.0, effective July 2026. §6 Domain 1 and detailed objectives (p. 4), Domains 2 and 7 (pp. 4–5), §7 How to Prepare (p. 6). Official download: https://everpath-course-content.s3-accelerate.amazonaws.com/instructor/6nizmqk8tpzpfjvt6qmmav7rh/public/1783542847/Claude+Certified+Associate+%E2%80%93+Foun
- **Anthropic's official CCAO-F prep course,** Course 2, *Prompting & Task Execution* (public course page: learning objectives; path page summary).
- **Claude Platform Docs,** *Prompt engineering overview* (success criteria, ways to test, a first draft; not every failure is a prompt problem): https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview
- **Claude Platform Docs,** *Prompting best practices* (explain why; Claude generalizes from the explanation; say what to do instead of what not to do): https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices
- **Claude blog,** *Best practices for prompt engineering* (November 10, 2025: troubleshooting common prompt issues; start simple and test each addition): https://claude.com/blog/best-practices-for-prompt-engineering

- **Claude Academy, Claude 101,** *Getting better results*, "The iteration mindset": https://academy.claude.com/courses/claude-101/getting-better-results
- **Claude Academy, AI Fluency: Framework and foundations,** *The Description-Discernment loop*: https://academy.claude.com/courses/ai-fluency-framework-foundations/the-description-discernment-loop
- **Anthropic,** *Claude Opus 4 and 4.1 can now end a rare subset of conversations* (August 15, 2025: editing and retrying previous messages create new branches): https://www.anthropic.com/research/end-subset-conversations
- **Claude Help Center,** *How do usage and length limits work?* (start a new conversation when a long chat nears its limit): https://support.claude.com/en/articles/11647753-how-do-usage-and-length-limits-work
- **The app itself:** Claude Desktop on Linux, Chat view, Sonnet 5.5 Medium, incognito chats, observed 2026-10-05 (the Edit pencil and its branch note, the version arrows, the copy control). Menus, labels and status lines can change between versions.

The checklist, the five-step loop, "one change per run", the symptom map and the edit / follow-up / start-fresh rule are this course's own teaching tools, built from the sources above. The hands-on material is real: this course's own published Lesson 2.2 notes. The checklist and every scoring decision are the course author's.

What you see in the lesson is a real account. Settings, Account and Memory were never opened. A usage notice that Claude Desktop showed during recording is blurred.